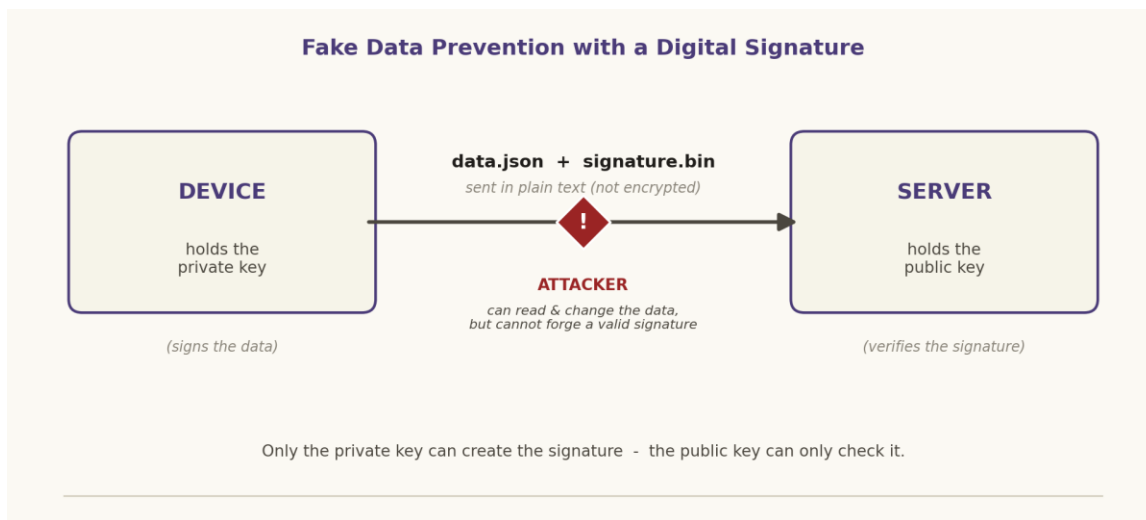




Università
degli Studi di
Messina

Fake Data Prevention with Digital Signatures

Proving that data sent from an edge device to a central server is genuine and untampered, using a single security tool: an RSA digital signature (RSA-2048, PKCS#1 v1.5, SHA-256)



Shayan Taghizadehjavid

Matricola: 562634

Course: System Security

Prof. Villari & Prof. Ruggeri



Abstract

This project shows how a single digital signature can stop fake or tampered data from being accepted by a server. A simulated edge device builds a small JSON sensor reading, signs the exact message bytes with its RSA private key, and sends the message together with its signature in plain text. The receiver checks the signature with the matching public key and accepts the data only if the check passes. A man-in-the-middle attacker intercepts the plain-text message and changes the sensor value to a fake number, but cannot produce a matching signature because it does not hold the private key, so the receiver rejects the message. The project is honest about its scope: a digital signature provides authenticity and integrity, not confidentiality. This is exactly what the goal "fake data prevention" requires, and the report explains why one signature is enough.

1. Introduction and Overview

Many systems trust the data they receive without checking where it came from. An edge device, for example a temperature sensor, sends a reading to a central server, and the server stores or acts on that reading. If an attacker sits on the network between the two, the attacker can change the reading before it arrives. The server has no obvious way to know the data is fake. This is the problem the project addresses: fake or tampered data being accepted as if it were genuine.

Two properties matter here. Integrity means the data has not been changed after it left the device. Authenticity means the data really was produced by the device we think it was, and not by someone pretending to be that device. If both hold, the server can trust the reading. If either fails, the server should reject it.

The tool used to provide these two properties is a digital signature. The device signs its message with a private key that only it holds. The server checks the signature with a matching public key that anyone may hold. Because only the private key can create a valid signature, a correct signature proves that the holder of the private key produced this exact message. Because the signature is computed from the exact bytes of the message, any change to the message makes the signature stop matching. So a passing check proves both authenticity and integrity at the same time.



This report is honest about what the project does and does not do. A digital signature gives authenticity and integrity. It does not give confidentiality. The message is sent in plain text and anyone on the network can read it. The project does not try to hide the data, because the goal is to stop fake data, not to keep the data secret. Section 5 explains why this scope is the correct match for the stated goal.

The whole demo is built around one security tool only: an RSA-2048 digital signature using PKCS#1 v1.5 padding and a SHA-256 hash. There is no encryption, no token, no certificate. Keeping the design to a single mechanism makes it easy to explain and easy to defend.

2. System Architecture

The project is organised as four steps that run in order: setup, sender, hacker, and receiver. The same logic is available in two forms: a set of small command-line scripts, and a single-page web demo. Both call the same underlying functions, so the behaviour is identical no matter how it is run.

2.1 Files and Project Layout

The project folder contains the four step scripts, a shared core module, a small Flask web application, and the web page assets. The layout below shows the files that matter for understanding the project.

- `setup_keys.py` - Step 1. Generates the RSA-2048 key pair and writes `private_key.pem` and `public_key.pem`.
- `sender.py` - Step 2. Builds the JSON message, signs it with the private key, and writes `data.json` and `signature.bin`.
- `hacker.py` - Step 3. The attacker. Reads `data.json`, changes the value to 999, writes it back, and leaves the old `signature.bin` in place.
- `receiver.py` - Step 4. Loads `data.json`, `signature.bin` and `public_key.pem`, verifies the signature, and prints `ACCEPTED` or `REJECTED`.
- `core.py` - The same four steps as plain functions (`setup_keys`, `run_sender`, `run_hacker`, `run_receiver`), plus `project_state` and `reset` helpers. The scripts and the web app both use this.
- `app.py` - The Flask web server. Routes: `/` (page), `/api/state`, `/api/setup`, `/api/send`, `/api/hack`, `/api/receive`, `/api/reset`. Runs on `http://127.0.0.1:5000/`.
- `templates/index.html` - The web page (the lab-report style interface).



- static/app.js - The page logic that calls the API and prints the output of each step.
- static/style.css - The page styling.
- requirements.txt - The two dependencies: flask and cryptography.
- run.bat - A Windows launcher that finds Python, installs the dependencies, starts the server, and opens the browser.

2.2 The Four Steps

The four steps below are the heart of the project. Running them in order reproduces both the honest case and the attack case.

Step 1, setup (setup_keys.py). This generates a fresh RSA-2048 private key with public exponent 65537, then derives the matching public key from it. The private key is saved to private_key.pem in PKCS8 format with no password. The public key is saved to public_key.pem in SubjectPublicKeyInfo format. The private key is the secret that must stay on the device; the public key is meant to be shared with the server. No certificate is created, because the project does not need a third party to vouch for the key.

Step 2, sender (sender.py). The device builds a small JSON message: a temperature reading with a value, a unit, and a device id. For example: {"sensor": "temperature", "value": 25, "unit": "Celsius", "device_id": "iot-device-001"}. It loads the private key, then signs the exact bytes of the message. It writes two files: data.json, which is the message in plain text, and signature.bin, which is the signature of that message. Both files are what gets sent across the network.

Step 3, hacker (hacker.py). This plays the attacker who has intercepted the message. Because the message is plain text, the attacker can read it and change it. The attack does one thing: it reads data.json, changes value from 25 to 999, and writes the file back. The attacker leaves signature.bin untouched, because making a new signature that matches the changed data would need the private key, which the attacker does not have.

Step 4, receiver (receiver.py). The server reads data.json and signature.bin, loads public_key.pem, and verifies the signature against the message. If the signature matches, the data is genuine and the server prints the reading and accepts it. If it does not match, the cryptography library raises InvalidSignature, and the server prints a warning and rejects the message.



2.3 The Web Interface

The web interface presents the same four steps as a single scrolling page with the look of a paper lab report. Each step has a button and a small terminal-style box that shows the output of that step. The page is served by Flask from `app.py` and driven by `static/app.js`, which calls the API routes and prints the returned log lines.

There is a button for each step: Run `setup_keys.py`, Run `sender.py`, the attacker controls, and Run `receiver.py`. The receiver section ends with a clear verdict box that turns green and says 'message accepted' when the signature is valid, or red and says 'message rejected' when it is not. The attacker section has two buttons: one runs the attack and tampers with the data, and one skips the attack and forwards the message untouched, so the honest path and the attack path can both be shown.

There is also a 'Run all phases in order' button that runs setup, sender, attacker, and receiver back to back, and a 'Reset working files' button that deletes the generated files so the demo can be run again from a clean state. The reset and state routes let the page show which files currently exist on disk.

3. Cryptographic Design

The project uses one cryptographic mechanism: an RSA digital signature. This section explains the key pair, the signing process, and the verification process, and why the asymmetric design is what makes the whole scheme work.

3.1 The RSA Key Pair

RSA is an asymmetric algorithm, which means it uses two different but mathematically linked keys. In this project the key size is 2048 bits and the public exponent is 65537, which are the standard, safe choices.

The two keys have different jobs, and this split is the core idea of the project.

- Private key - secret, kept on the device. It is the only key that can create a valid signature. In the project it is stored in `private_key.pem`.
- Public key - shared, given to the server. It can check a signature but can never create one. In the project it is stored in `public_key.pem`.



3.2 The Signing Process

Signing happens in the sender step and works in two parts: hash, then sign.

First the message bytes are passed through the SHA-256 hash function, which produces a fixed 256-bit fingerprint of the message. Any change to the message, even a single character, produces a completely different fingerprint. Hashing first also lets the signature cover a message of any length.

Second, the RSA private key signs that hash using PKCS#1 v1.5 padding. The output is the signature, a block of bytes that is saved to signature.bin. In code this is a single call: `private_key.sign(message_bytes, PKCS1v15(), SHA256())`. Only the holder of the private key can produce this signature for this message.

The message is then sent in plain text alongside the signature. The project does not encrypt the message, because the goal is to prove the data is real, not to hide it.

3.3 The Verification Process

Verification happens in the receiver step and reverses the logic with the public key. The receiver hashes the message it received with SHA-256, then asks the public key whether the signature matches that hash. In code this is a single call: `public_key.verify(signature, message_bytes, PKCS1v15(), SHA256())`.

There are two possible outcomes. If the signature was made by the matching private key over this exact message, the check passes and the receiver accepts the data. If the message was changed, or the signature was made by a different key, the hashes no longer line up, the library raises `InvalidSignature`, and the receiver rejects the data.

Why does this work? The maths of RSA makes the private and public keys a one-way pair. The private key can produce a signature that the public key can confirm, but knowing the public key does not let anyone work backwards to the private key, and the public key cannot itself create a signature. So a signature that the public key accepts could only have come from the private key. For example: the server holds the public key and can check thousands of messages, but it still cannot forge even one signature, and neither can an attacker who copies the public key off the wire.



4. The Attack and Why It Fails

To show that the signature actually protects the data, the project includes one attacker, playing a man-in-the-middle on the network. The honest path is: setup, then sender, then receiver, and the result is ACCEPTED. The attack path inserts the hacker step between sender and receiver.

The attack is concrete and simple. The message travels in plain text, so the attacker can read data.json and edit it. The attacker opens the file, changes value from 25 to 999, and saves it. The goal is to make the server store a fake temperature reading and treat it as genuine.

The problem for the attacker is the signature. signature.bin was created over the original message. To make the receiver accept the fake reading, the attacker would need a new signature that matches the changed message. Creating that signature requires the device's private key, and the attacker does not have it. The private key never leaves the device; only the public key is shared. So the attacker is stuck: they can change the data, or they can keep the matching signature, but they cannot do both.

In the demo the attacker changes the data and leaves the old signature in place, hoping the server does not check. When the receiver runs, it hashes the tampered message and compares it against the old signature. The two no longer match, the library raises InvalidSignature, and the receiver rejects the message. In the web demo the result is reported as 'message rejected' with the reason 'bad_signature'; the command-line receiver prints a security warning and stops. The fake reading of 999 never reaches the application.

It is worth being precise about what the attacker can still do. They can read the data, since it is not encrypted. They can delete or block the message, which is denial of service and is outside the scope of this project. What they cannot do is the one thing that matters for this goal: make the server accept fake data as genuine.



5. Security Analysis

This section states plainly what the project protects, what it does not protect, and why a single signature is enough for the stated goal.

5.1 Integrity

Integrity means the data was not changed after it was signed. The signature is computed from the exact bytes of the message through SHA-256. If even one character of data.json changes, its hash changes, and the old signature no longer matches. The receiver detects this and rejects the message. The attack in Section 4 is exactly this case: the value changes from 25 to 999, and the receiver catches it. So the project provides integrity.

5.2 Authenticity

Authenticity means the data really came from the expected device. Only the device holds the private key, and only the private key can create a signature that the public key accepts. So when verification passes, the server knows the message was signed by the holder of that private key and not by anyone else. An attacker who only has the public key, which is freely shared, cannot produce a valid signature for their own fake message. So the project provides authenticity.

5.3 What Is Not Protected: Confidentiality

The project does not provide confidentiality, and this is by design. The message is sent in plain text in data.json, so anyone who can see the network traffic can read the sensor reading. A digital signature is not encryption; it proves that data is genuine, it does not hide it.

This is an honest limitation, not a bug. If the goal were to keep the reading secret, the project would need encryption as well, for example a symmetric cipher over the message. But the stated goal is fake data prevention, which is about trust in the data, not secrecy of the data. Claiming confidentiality here would be wrong, and the project does not claim it.

For completeness, the project also does not defend against an attacker who simply blocks or deletes the message (denial of service), and it does not include a freshness mechanism such as a timestamp or nonce, so an old signed message could in principle be replayed. These are outside the scope of the single-signature goal and would be the natural next steps.

5.4 Why a Single Signature Is Sufficient

The goal of the project is narrow and clear: stop the server from accepting fake or tampered data. That goal is exactly the pair of properties a digital signature gives, authenticity and



integrity. Adding more tools would not make fake data prevention any stronger; it would only address a different goal, such as secrecy.

Keeping the design to one mechanism also has practical value. There is one thing to get right, one thing to explain, and one thing to defend in an exam. The asymmetric split, private key signs and public key verifies, is enough to make forging a valid signature without the private key infeasible. For the question 'is this data real?', a single RSA signature answers it completely, and the demo proves it: the honest message is accepted, and the tampered message is rejected.

6. Conclusion

This project shows that a single RSA digital signature is enough to prevent fake data from being accepted. The device signs each message with its private key, and the server checks the signature with the public key. Because only the private key can create a valid signature, and because the signature is tied to the exact bytes of the message, the server can be sure the data is both genuine and unchanged.

The man-in-the-middle attack makes the point concrete. The attacker can read and edit the plain-text message, but cannot forge a matching signature without the private key, so the receiver detects the change and rejects it. The honest message is accepted with its original value of 25; the tampered message with the fake value of 999 is rejected.

The project is deliberately honest about its scope. It provides authenticity and integrity, not confidentiality, and that is the correct match for the goal of fake data prevention. Possible extensions, such as adding encryption for secrecy, or a timestamp to stop replay, would build on the same key pair without changing the core idea that only the holder of the private key can sign.